

Same Substrate State, Same Content: Guarantee A as Standalone Read-Determinism Commitment in the Coordination Knowledge Substrate Pattern

Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

May 5, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the first of the five guarantees the determinism contract names — *Guarantee A: same substrate state produces same content for any read* — as a standalone architectural commitment with independent operational content, separable from the integrating determinism contract, the substrate/model representation distinction, the four sibling guarantees, the allowed non-determinism categories, and regression testing.

Abstract

The CKS determinism contract (§4.1, §6.2, §11.3 of the source paper) names five guarantees governing how substrate state, orchestration rules, and cell behavior compose. The first — that the same substrate state produces the same content for any read — is the read-side commitment that makes substrate’s authority architecturally usable. Without it, every other commitment that depends on reading substrate (source-of-truth, path retraceability, the AI-as-substrate-mediator’s read role, the human inspect right, regression testing) operates on an ambiguous input. This note formalizes Guarantee A as a standalone architectural commitment, separable from the integrating frame (A2.55), the substrate/model boundary (A2.56), the four sibling guarantees (A2.58–A2.61), allowed non-determinism (A2.62), and regression testing (A2.63). It states the four operational components Guarantee A requires — read consistency across time, across readers, across access paths, and across replicas — clarifies what the commitment does not require, distinguishes it from four adjacent read-consistency patterns commonly conflated with it, identifies the commitments it is load-bearing for, names the failure modes that violate it, and provides an operational test for whether a system instantiates it.

1. Why Guarantee A needs to be formalized as standalone

The parent note A1.10 commits to the determinism contract as a whole. The integrating-frame note A2.55 enumerates the five guarantees. A2.56 formalizes the substrate/model boundary within which the substrate-side guarantees operate. This note specializes the first of those guarantees — that the same substrate state produces the same content for any read — as having independent architectural content, with particular weight on the read interface as the operationally pervasive boundary of substrate.

The motivating cases share a common shape: a reader observes substrate at moment t and must see substrate’s actual state at t , not a derivation, transformation, or stale view. This applies to a human inspecting substrate, a cell reading substrate to make a decision, a regression test verifying state, and an external derivative view per Pattern B of A1.16. Each requires read-determinism per Guarantee A; without it, the reader’s basis for inference, decision, or verification is ambiguous.

The strategic prior-art posture follows. Guarantee A forecloses architectures where substrate reads return varying content depending on which reader, which access path, or which replica is consulted. Patentable derivations addressing AI-coordination architectures with read-consistency properties, deterministic-AI-state architectures, or replayable-AI-coordination systems are substantially more contestable when Guarantee A is publicly formalized as standalone prior art.

The relationship to A2.56 is also relevant. A2.56 commits to representation determinism across reads as one component of substrate-side determinism; this note specializes that component with the four operational sub-components specific to read operations and the failure-mode landscape that read interfaces are particularly vulnerable to in modern data architectures.

2. The Guarantee A commitment, defined precisely

In the CKS pattern, a system satisfies **Guarantee A** when reads against substrate return content equivalent to substrate’s canonical content across four operational components. By *S*’s *canonical content* this note means the content substrate state *S* architecturally carries — its identifiers, relationships, field values, and provenance metadata — as substrate represents them per A2.08. The four components specify the dimensions across which read content must remain equivalent to canonical content; a system failing any one fails the guarantee, regardless of how robustly the other three are satisfied.

(a) Read consistency across time. For a substrate state *S* that persists across a time interval $[t_0, t_1]$, reads against *S* at any time within that interval return content equivalent to *S*’s canonical content. Within the persistence interval, reads do not see varying transformations or derivations that would make different reading times produce different content. Transitions between intervals — that is, state changes — are governed by Guarantee C (A2.59), not by Guarantee A.

(b) Read consistency across readers. For a substrate state *S* and any two readers R_1, R_2 reading *S* at the same moment, R_1 and R_2 see content equivalent to *S*’s canonical content. The substrate does not present different views to different readers based on identity, role, or access pattern. The component holds subject only to authority-structure-based access rights per A2.47, which gates whether a reader can read at all — not what content is returned for permitted reads.

(c) Read consistency across access paths. For a substrate state *S* and any access paths P_1, P_2 (different APIs, different query interfaces, different field accessors) that retrieve content from *S*, the access paths return content equivalent to *S*’s canonical content. A path that returns transformed, summarized, or filtered content is operating as a derivative view per Pattern B of A1.16, not as a substrate read; substrate’s reads themselves do not vary by path.

(d) Read consistency across replicas. For a substrate state *S* that may be physically replicated across multiple storage backends, reads against any replica return content equivalent to *S*’s canonical content. Replication is an operational concern; the architectural commitment is that all replicas present the same canonical con-

tent. Implementations may use synchronous replication, consensus protocols, snapshot isolation, or other mechanisms to satisfy this component.

3. What the guarantee does NOT claim

The standalone treatment is not maximalist. Stating precisely what the guarantee does not claim keeps the framing from drifting beyond what the source paper supports.

It does not claim reads are physically free or arbitrarily fast. The architectural commitment is to read-content determinism, not to read-performance properties; cost and latency are deployment concerns separable from the guarantee.

It does not claim substrate state is itself static. Substrate state evolves as writes occur per A2.10. What Guarantee A specifies is that for any substrate state S — whatever S is at any moment — reads against S return S 's canonical content. State changes are governed by Guarantee C (A2.59); Guarantee A is about read-content determinism *given* a state.

It does not require all reads to traverse the same physical path. Caches, indexes, materialized projections, and replicas may exist as operational infrastructure, provided each presents content equivalent to substrate's canonical content. Read paths may be optimized for performance; architectural read content is determined by substrate state.

It does not specify implementation patterns for read consistency. Implementations may use snapshot isolation, multiversion concurrency control, read-after-write consistency, eventual-consistency-with-synchronous-read-repair, or other mechanisms. The architectural commitment is to the components themselves; specific implementations are deployment choices.

It does not foreclose access-control-based read filtering. A reader without authority for content C may be unable to read C at all per A2.47's authority structure. Access control gates which reads occur; it does not vary the content non-gated reads return.

It does not require substrate content to be human-readable in raw form. Substrate content has structured representation per A2.08, which may include encoded fields, identifiers, references, and metadata that require interpretation by tools. The human-readable commitment is per A2.25 (Requirement 2), separable from Guarantee A.

4. What the guarantee is NOT

Four adjacent read-consistency patterns are commonly conflated with Guarantee A. Each is a real and reasonable commitment in some other architecture; naming what Guarantee A is not is what prevents the misreading.

Not database transactional read consistency. Database read consistency is typically about read-write isolation under concurrent operations — ensuring a read sees a consistent snapshot of state while writes are concurrent against it. Guarantee A is broader: it specifies that the same substrate state produces the same content for any read, regardless of concurrency, across time, readers, access paths, and replicas. A system with strict serializability may still fail Guarantee A — for example, by personalizing read content per reader; a system without strict serializability may still satisfy Guarantee A, provided the four components are met

by other means.

Not eventual consistency with read repair. Eventually-consistent architectures permit read divergence across replicas, with read-repair mechanisms eventually converging replicas to a common state. Guarantee A is different: at any moment, reads against substrate state S return content equivalent to S 's canonical content for any reader, access path, or replica. Eventual consistency without synchronous read repair fails component (d).

Not materialized-view freshness. Materialized views are projections that may lag the source state, with freshness mechanisms updating views periodically or on demand. Guarantee A is different: substrate is the source, and reads against substrate return substrate's canonical content. Materialized views in adjacent components per Pattern B of A1.16 may be derivative; their freshness is a deployment concern about the derivative, not a substrate-determinism concern.

Not cache coherence. Cache coherence in computer architecture is the property that multiple caches present consistent views of shared memory. Guarantee A operates at a different layer: substrate's reads at the architectural-content level are deterministic regardless of underlying cache infrastructure. Cache coherence may be operationally relevant for implementations, but the architectural commitment is at the content level, not the cache-mechanism level.

5. Load-bearing connections to other CKS commitments

Guarantee A is load-bearing for several major CKS commitments.

The integrating determinism contract (A1.10). Guarantee A is one of the five guarantees the contract names; without read-determinism, the contract cannot operate at the read level, which is the operationally pervasive interface to substrate.

The source-of-truth commitment (A1.08, A2.42). Source-of-truth depends on substrate's authoritative content being deterministically readable. Without Guarantee A, source-of-truth queries could return varying content from the same state, and substrate's authority over its five categories (A2.43–A2.47) would be ambiguous in a way the source paper §11.3 explicitly rules out.

Path retraceability (A1.07, A2.41). Path reconstruction traverses substrate content through reads at each step; Guarantee A is what makes each step deterministic. Without it, paths reconstructed at different times or by different readers diverge — failing retraceability not because the path content is wrong but because the reads producing the reconstruction are inconsistent.

The AI-as-substrate-mediator commitment (A1.04, Property A from A2.19). The LLM reads substrate as the primary source of state; Guarantee A is what makes this read deterministic, so the same prompt against the same substrate state yields consistent LLM behavior on the substrate-side input.

The human inspect right (A1.01, A2.01). Humans exercising inspect rights see substrate content deterministically per Guarantee A. Without it, different inspections of the same state return different content, and the architectural authority the inspect right names becomes architecturally hollow.

6. Failure modes that violate the guarantee

The first four failure modes correspond directly to the operational components in §2; the remaining six are mechanism-specific anti-patterns common in modern data architectures.

Component-orthogonal failures.

(a) Time-varying reads. Reads against substrate state S return different content at different times even though S has not changed — typically caused by caching that introduces varying staleness, derivation that varies with computation context, or storage backends producing time-varying output for reasons unrelated to substrate state.

(b) Reader-specific reads. Reads return different content depending on reader identity, role, or access pattern, beyond authority-structure-based access control. Personalization, role-based content transformation, and A/B-style content variation are common operational features but architecturally fail component (b) when applied to substrate’s authoritative content.

(c) Access-path-divergent reads. Different access paths to the same substrate state return different content because different APIs or query interfaces apply different transformations, format conversions, or summarizations.

(d) Replica-divergent reads. Different replicas of substrate state return different content at the same moment — common in eventually-consistent storage architectures without synchronous read repair.

Mechanism-specific anti-patterns.

(e) Derivation-substituted reads. Reads return derivations, summaries, or transformations of substrate content rather than canonical content. Substrate’s own reads must return canonical content; derivative views are identified separately per Pattern B of A1.16.

(f) Format-converted reads. Reads return content in formats that vary by operational context (request format, content-negotiation outcome). The content beneath may be deterministic, but if the interface returns different field structures or different identifiers across calls, the interface fails Guarantee A.

(g) Eventual-consistency-without-repair. Substrate is operated in an eventually-consistent architecture without synchronous read repair; reads return varying content depending on which replica responds. Eventual convergence does not satisfy Guarantee A, which specifies determinism at any given read moment.

(h) Cache-stale reads. Caches serve content past the underlying substrate state’s persistence interval; reads against the cache return content that does not match canonical content for the current state, failing component (a) at the interface level.

(i) Lazy-derivation reads. Substrate content is derived lazily at read time through computation that varies across read events — due to computational non-determinism, dependence on external state, or context-sensitive logic — so derived content varies even though substrate state has not changed.

(j) Read-time non-deterministic transformations. Transformations applied at read time introduce non-determinism — random ordering of result sets without explicit sort, sampling-based summarization, time-dependent rendering — so reads vary in operationally observable ways across calls against the same substrate state.

7. Operational test

A system satisfies Guarantee A if and only if all of the following hold at all times during the substrate’s existence.

1. For any substrate state S and any time interval $[t_0, t_1]$ during which S persists, reads against S at any time within $[t_0, t_1]$ return content equivalent to S ’s canonical content (read consistency across time).
2. For any substrate state S and any two readers R_1, R_2 reading S at the same moment within their authorized scopes, R_1 and R_2 see content equivalent to S ’s canonical content (read consistency across readers).
3. For any substrate state S and any access paths P_1, P_2 retrieving content from S , P_1 and P_2 return content equivalent to S ’s canonical content (read consistency across access paths).
4. For any substrate state S replicated across multiple storage backends or replicas, reads against any replica return content equivalent to S ’s canonical content (read consistency across replicas).
5. Read-content equivalence does not depend on operational context beyond authority-structure-based access control per A2.47. A read either occurs (returning deterministic content) or is gated by access control (returning no content); there is no third category in which the same read returns different content based on context.
6. Where read paths involve transformations, derivations, or rendering, those transformations are themselves deterministic, or the path is explicitly identified as a derivative view per Pattern B of A1.16 with substrate’s canonical content remaining the authoritative referent.

A system that fails any of (1)–(6) does not satisfy Guarantee A in the architectural sense, regardless of how well its read operations function in typical deployment.

8. Why naming Guarantee A as standalone matters

Implementations under pressure to integrate with enterprise infrastructure consistently drift toward read architectures that introduce non-determinism. The drift is steady because read non-determinism is operationally common in current data architectures — eventual consistency, cache layers, materialized views, personalization, content negotiation — and rhetorically accessible: audiences understand “the system might return slightly different content depending on context” more easily than “the system must return the same content for the same state.”

Implementations that drift away from Guarantee A produce systems where substrate reads are architecturally ambiguous: the same substrate state can produce different content depending on operational context. The downstream consequences manifest as source-of-truth fragmentation (A1.08 fails because substrate authority becomes ambiguous), path-retraceability failures (reconstructed paths diverge across reads), AI-mediator failures (the LLM operates on varying content for the same substrate state), and inspect-right compromise (humans see varying content when inspecting the same state).

Naming Guarantee A as a standalone architectural commitment gives downstream readers a precise specification of what read-determinism the architecture requires. The sibling specialization notes A2.58–A2.61 formalize the other four guarantees; A2.62 the allowed non-determinism categories; A2.63 regression testing. Together they give the full operational decomposition of A1.10’s determinism contract.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Same Substrate State, Same Content: Guarantee A as Standalone Read-Determinism Commitment in the Coordination Knowledge Substrate Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.